

Avionics Interface Control Document

DOCUMENT NUMBER	REVISION	RELEASE DATE
BLEP-01	**A**	**10/29/2026**

CONTENT OWNER:
Jordan Gamache

Copyright © 2024, 2026 Jordan Gamache. All rights reserved.

Table of Contents

1. Introduction....2
2. Glossary....3
3. Software....4
   3.1. Software Development....4
   3.2. Operational Requirements....4
4. Hardware....5
   4.1. MPU Hardware....5
      4.1.1. Teensy 3.2 + Adafruit FeatherWing Adapter....5
      4.1.2. Sensors....6
      4.1.3. Radio Communication....6
   4.2. SPU Hardware....6
      4.2.1. Adafruit Feather RP2040....6
5. Payload Data Transmit Protocol....7

1. Introduction

The purpose of this document is to outline the specification of the design and function of the BLEP Avionics system. BLEP is an integrated modular avionics system that performs a series of operations on shared hardware. These tasks including recording sensor data, calculating state values, correcting moments of rotation during ascent, communicating over various protocols, and deploying a parachute to support safely returning the vehicle to the ground.

The following document describes different aspects of the design, including the software and hardware designs, and outlines the network communication protocol. All decisions herein were made after significant research was conducted; however, designs improvements, hardware optimizations, and software development will continue to be implemented as time permits.

2. Glossary

COTS	Commercial Off The Shelf
CRC	Cyclic Redundancy Check
DOF	Degrees Of Freedom
FCSN	Frame Check Sequence Number
FSB	Functional Status Byte
FSW	Flight Software
I/O	Input/Output
I^2C	Inter-Integrated Circuit
IDE	Integrated Development Environment
MPU	Main Processing Unit
PCB	Printed Circuit Board
PDTP	Payload Data Transmit Protocol
PWM	Pulse Width Modulation
RAM	Random Access Memory
RCS	Rotation Control System
SN	Sequence Number
SPI	Serial Peripheral Interface
SPU	Secondary Processing Unit

3. Software

The flight software that governs the operation, state, and behavior of the avionics system is written in C (Embedded C), which is then compiled by the Arduino Integrated Development Environment (IDE) and programmed into the on-chip flash memory of the target hardware element(s).

3.1. Software Development

The flight software (FSW) is written in C and leverages the use of multiple support header files to separate functionality by context. The main flight program can be customized by setting the value of configuration variables in the “flightConfiguration” header file.

3.2. Operational Requirements

FSW-1	The [FSW] shall provide error handling and must be capable of clearing the command queue in the event of an error.
FSW-2	The [FSW] shall log all data to a removable disk for redundancy.
FSW-3	The [FSW] shall provide a method for pre-flight configuration.
FSW-4	The [FSW] shall convert data to be transmitted in accordance with Section 4 of this document [PDTP].
FSW-5	The [FSW] shall support the manual override of key events during the flight plan.
FSW-6	The [FSW] shall provide audio and visual cues to indicate vehicle state.
FSW-7	The [FSW] shall be able to detect when apogee has been reached and must be able to independently deploy the parachute(s).

4. Hardware

The avionics system was designed to integrate multiple Commercial Off-The Shelf (COTS) hardware elements to reduce costs while mainting the ability to support the Operational Requirements outlined in Section 3.2. The avionics system contains two main elements, the Main Processing Unit (MPU) and the Secondary Processing Unit (SPU).

4.1. MPU Hardware

The MPU leverages the Adafruit Feather family of microcontrollers due to the uniform size and pin arrangement. The backbone of the MPU is an “Adafruit Quad 2x2 FeatherWing”, a prototyping PCB board that can support connecting 4 Adafruit Feather “wings” in a 2-by-2 grid, with a grid of additional “proto(typing)” holes. These copper-lined development holes allow for breakout boards, sensors, and wires to be soldered in the space between the FeatherWing pins, increasing functionality and flexibility.

The MPU is powered by a Teensy 3.2 microcontroller connecteded to a FeatherWing adapter, a RFM69 900MHz radio FeatherWing breakout board, and various sensors used to capture environmental data to calculate and process attributes regarding the vehicle state.

4.1.1. Teensy 3.2 + Adafruit FeatherWing Adapter

The Teensy 3.2 microcontroller is powered by an ARM Cortex-M4 with a clock speed of 72 MHz, 256KB of on-chip Flash memory, and 64KB of RAM. With regards to inputs and outputs (I/O), the Teensy 3.2 has 34 digital input/output pins, 12 of which are Pulse Width Modulation (PWM) output pins, 21 analog input pins and 1 analog output pin. It also has 3 serial ports, 1 SPI port, and 2 I^2C ports. The vast amount of I/O support is precisely why this microcontroller was selected to power the MPU, and the Adafruit FeatherWing Adapter provided the ability to easily slot this board onto the MPU backplane, directly linking the SPI and I^2C lines to common buses.

4.1.2. Sensors

The MPU utilizes two primary sensors to create a general “space-state” of the vehicle. The BME280 temperature sensor and the MPU6050 6-DOF sensor are used in parallel to determine attributes such as vehicle orientation, altitude, rotation, acceleration, temperature, and pressure. The discrete values are read via the I^2C bus and processed by the FSW.

4.1.3. Radio Communication

The MPU (and SPU) use an Adafruit RFM69 900MHz “RadioFruit” FeatherWing radio module to send “packetized” data over radio. They support roughly a kilometer of range, and are configured by the FSW. Data packets sent using these modules are encrypted with AES-128 encryption.

4.2. SPU Hardware

Like its counterpart, the SPU utilizes the Adafruit Feather family of microcontrollers, with an “Adafruit FeatherWing Doubler”, a 2-by-1 prototyping PCB board serving as its backplane. The SPU does not communicate with any sensors, but uses the same Radio Module outlined in the previous Section (4.1.3) to communicate with the MPU.

4.2.1. Adafruit Feather RP2040

The SPU is powered by an Adafruit Feather RP2040 microcontroller, a chip designed and manufactured by Raspberry Pi and built into an Adafruit Feather breakout board. The RP2040 has a 32-bit Cortex M0+ dual core microprocessor, with a clock speed of 125 MHz. It has a surprising 21 I/O pins, 16 of which support PWM outputs, and has 264 KB RAM.

5. Payload Data Transmit Protocol

The FSW implements a custom protocol to convert sensor data into an encoded network packet for transmission, typically over radio, called the Payload Data Transmit Protocol (PDTP).

PDTP packets are 32 bytes wide, with 26 bytes of payload data encapsulated by 3 “header” bytes and 3 “footer” bytes. The packet headers consist of the Functional Status Byte, Destination Address, and Source Address, while the packet footers contain the packet’s Sequence Number and the Frame Check Sequence Number, for data integrity.

Functional Status Byte	**Destination Address**	**Source Address**	**Payload**	**Sequence Number**	**Frame Check Sequence Number**
1 byte	1 byte	1 byte	26 bytes	1 byte	2 bytes

Functional Status Byte

The functional status byte (FSB) defines the message classification, and can be set to ‘INFO’, ‘DATA’, ‘ACK’, or ‘ERROR’. The upper 4-bits of the FSB define the “functional status”, as defined by the options below, while the lower 4-bits of the FSB define the pad size, such that: `[1 byte (8 bits)] => [4 bits][4 bits] => [status][pad]`.

0 - *INFO*: Info messages are text-only (i.e. no sensor data) and contain information related to vehicle state, events, or system health. The maximum pad size is 15.

1 - *DATA*: Data messages contain sensor data, and follow the exact encoding defined in the “payload” section. Data messages do not support padding, as the full 26-bytes of payload data are utilized.

2 - *ACK*: Taken from the TCP “SYN/ACK/SYNACK” concept utilized by the Transmission Control Protocol’s (TCP) 3-Way Handshake, the ACK, or “acknowledge(d)” status type is used for maintaining connection between

remote devices and serve as the “heart beat” for the protocol. The maximum pad size is 15. These messages are considered “lower priority” than their counterparts, and are only sent if a message of a different status has not been sent in a predefined amount of time.

3 - *ERROR*: If an error is detected in the operational flight program, the global error flag is set, and PDTP will transmit all DATA messages as ERROR messages instead, to indicate to the recipient that an error has occurred. The payload data contained in this type of message is identical to the DATA message type, and does not support padding.

Destination Address
This field helps designate the flow of network traffic. Values of **115** indicate that the intended *destination* of the data packet is the MPU, and values of **130** indicate that the intended *destination* of the data packet is the SPU.

Source Address
This field helps designate the flow of network traffic. Values of **115** indicate that the *source* of the data packet was the MPU, and values of **130** indicate that the *source* of the data packet was the SPU.

Payload
The data payload is encoded into 26 bytes, with optional padding as needed. The format for the data payload is as follows:

Bytes	Description	Data Type	Bytes
1-2	Temperature	integer	2
3-4	Pressure	integer	2
5-6	Altitude	integer	2
7-8	Humidity	integer	2
9-10	Yaw	integer	2
11-12	Pitch	integer	2
13-14	Roll	integer	2

15-16	Accel_X	integer	2
17-18	Accel_Y	integer	2
19-20	Accel_Z	integer	2
21-22	Yaw_Correction	integer	2
23-24	Pitch_Correction	integer	2
25-26	Roll_Correction	integer	2
TBD	Pad	TBD	TBD

Each data point will be converted into IEEE-754 Single-Precision Binary and sorted within the data message in the order shown in the table above. The data points are converted into integers by the FSW, which is stored in a 16-bit (or 2-byte) value, which supports a range of -32,768 to 32,768 (i.e. $|2^{15}|$), using 2's complement math to store the "sign bit".

Sequence Number

Each PDTP packet processed by the FSW increments the sequence number (SN), regardless of the data contained within. The SN ranges from 0-255, and is entirely for data integrity and analysis purposes, and can be used to identify dropped or discarded packets. The SN will reset after reaching its maximum of 255, and a sequence reset (as well as the number of resets) will be logged.

Frame Check Sequence Number

The Frame Check Sequence Number (FCSN) is a data integrity and validity check on the payload data values, utilizing a Cyclic Redundancy Check (CRC) error-detecting calculation which is then included in the PDTP packet itself. The FCSN is re-calculated on the receiver's end using the payload data received, and is compared against the FCSN included in the PDTP packet to ensure that the payload data was not corrupted during transit.